Travel to the Moon

1. Analisi Concettuale

1.1. Diagramma UML delle classi

1.1.1. Tipi di Dato

1.2.1. Specifiche delle classi

Operazioni ausigliarie

non_sovraposti(

Vincoli esterni

[V.Itinerario.ScaliNonSovraposti]

[V.Scalo.Relativo_arivoMinoreRelativa_partenza]

[V.Itinerario.ScaliOrdinati]

[V.Scalo.CrocieraEsistente_esiste_ScaloAssociato]

[V.Scalo.Primo_UltimoScalo]

[V.Crociera.NoOverbooking]

Operazioni di classe Cliente

eta(): Intero > 0

Operazioni di classe Crociera

data_fine_programmata(): Data

data_fine_effettiva(): DataOra

num_posti_liberi(): Intero >= 0

destinazioni(): DestinazioneScalo [0..*]

Operazioni di classe CrocieraLunaDiMiele

tradizionale(): Booleano

Operazioni di classe Itinerario

durata_programmata_giorni(): Intero > 0

Operazoni di classe cr_sc

effettivo_arrivo(): DataOra

effettiva_partenza(): DataOra

Operazioni di classe DestinazioneScalo

Operazioni ausiliarie

destinazione_esotica(): Boolean

crociere_in_periodo(d_in: DataOra, d_fin: DataOra): Crociera[0..*]

1.2. Diagramma UML degli Use-Case

1.2.1. Operazioni dello use-case Gestione prenotazioni

prenota_crociera(cl: Cliente, cr: Crociera, n: Intero > 0): Prenotazione

1.2.2. Operazioni dello use-case Statistiche

eta_media_clienti_in_destinazione_esotica(d_in: DataOra, d_fin: DataOra): Intero > 0

percentuale_destinazioni_gettonate(d_in: DataOra, d_fin: DataOra): Reale in 0..1

1.2.3. Operazioni dello use-case Gestione attività durante crociera

posti_da_vedere(dest: DestinazioneScalo): PostoEscursione[0..*]

2. Ristrutturazione per Basi Dati

2.1. Diagramma UML ristrutturato per bd

2.2. Specifica dei vincoli esterni

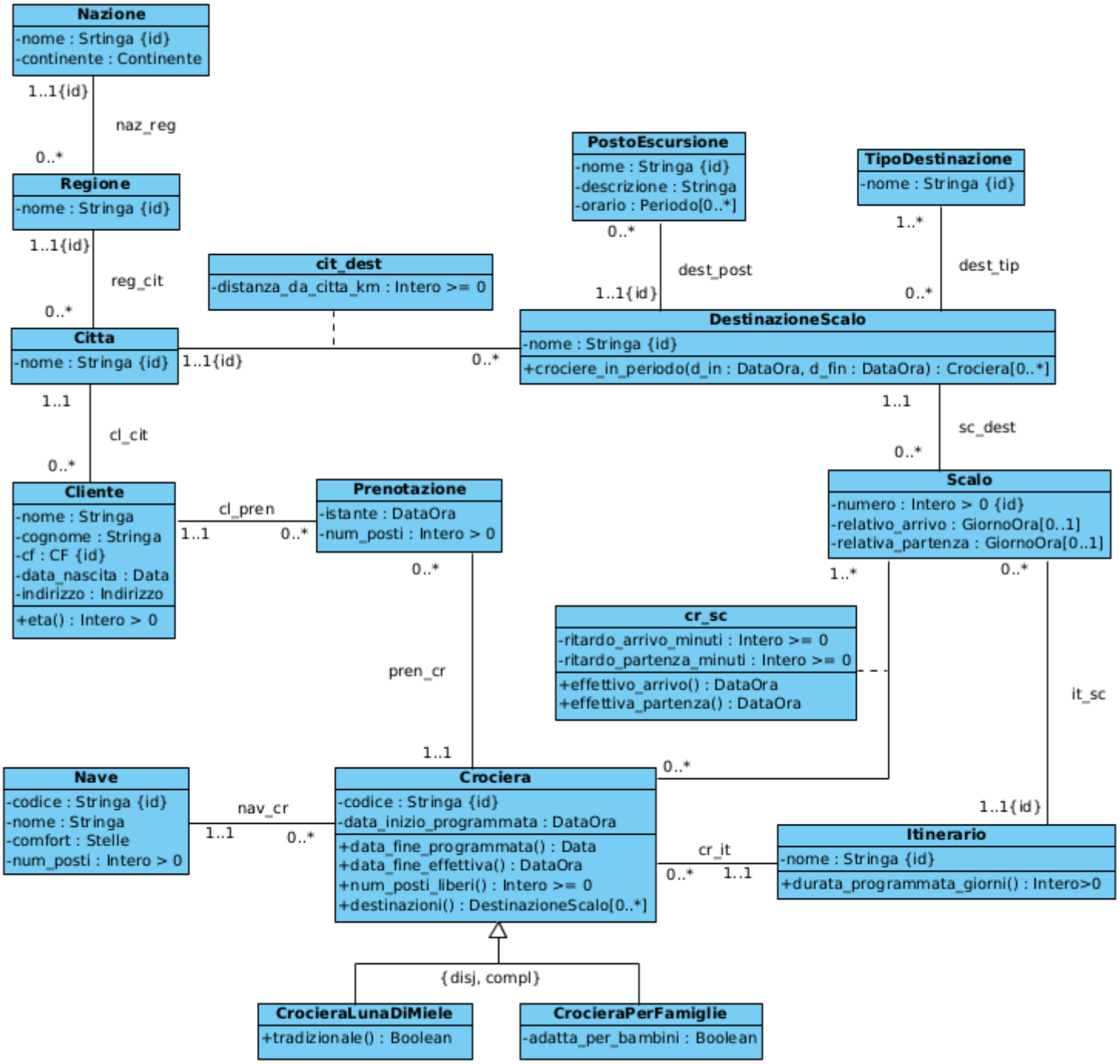
2.3. Traduzione diretta

2.3.1. Tipi di dato

2.3.2. Schema relazionale

1. Analisi Concettuale

1.1. Diagramma UML delle classi



1.1.1. Tipi di Dato

Tipo Indirizzo: tipo di record con campi

via: Stringa
civico: Stringa
cap: Stinga numerica di lunghezza 5 caratteri

Tipo CF: Stringa secondo lo standard

Tipo Stelle: Tipo intervallo di interi tra [3..5]

Tipo Continente: {Africa, America, Asia, Europa, Oceania, Antartide}

Tipo GiornoOra: Tipo record con campi
giorno: Intero >= 0
ora: Ora
Tipo Periodo: Tipo record con campi
data_iniz: Data
data_fin: Data
orario: OrarioGiorno[1..*]

Tipo OrarioGiorno: Tipo record con campi
giorno: {lunedi, martedi, mercoledi, giovedi,venerdi, sabato, domenica}
ora_inizio: Ora
ora_fine: Ora

Tipo Percentuale: Tipo record con campi
valore: Reale tra 0 e 1
simbolo: %

1.2.1. Specifiche delle classi

Operazioni ausigliarie

non_sovraposti(
d_in1: DataOra,
d_fin1: DataOra,
d_in2: DataOra,
d_fin2: DataOra
): Boolean

precondizione: nessuna
postcondizione:
- operazione non fa side-effect
- result = True
  - se e solo se
    - *d_fin1* < *d_in2* oppure *d_fin2* < *d_in1*

Vincoli esterni

[V.Itinerario.ScaliNonSovraposti]

Per ogni it: Itinerario, sc1: Scalo e sc2: Scalo
tali che
- *(it, sc1) : it_sc,*
- *(it, sc2): it_sc*
- *sc1 =/= sc2*

deve essere vero che
- non_sovrapposti(*sc1.relativo_arrivo, sc1.relativa_partenza, sc2.relativo_arrivo, sc2.relativa_partenza*) = True

[V.Scalo.Relativo_arivoMinoreRelativa_partenza]

Per ogni sc: Scalo deve essere vero che
- *sc.relativo_arrivo* non è valorizzato
  oppure
- *sc.relativo_arrivo* < *sc.relativa_partenza*
  oppure
- *sc.relativa_partenza* non è valorizzata

[V.Itinerario.ScaliOrdinati]

Per ogni *it*: Itinerario, per ogni *sc1*: Scalo e *sc2*: Scalo
tali che

- *(it, sc1): it_sc,*
- *(it, sc2): it_sc*
- *sc1 =/= sc2*

se

- *sc1.numero* < *sc2.numero*

deve essere vero che

- *sc1.relativo_arrivo* non è valorizzato
  oppure
- *sc1.relativo_arrivo* < *sc2.relativo_arrivo*
  oppure
- *sc1.relativa_partenza* non è valorizzato

[V.Scalo.CrocieraEsistente_esiste_ScaloAssociato]

Per ogni *sc*: Scalo, *it*: Itinerario

- tali che *(sc, it): it_sc*,

se esiste il link (c, sc): cr_sc
allora deve esistere it: itinerario e i link (c, i): cr_it e (it, sc): it_sc

[V.Scalo.Primo_UltimoScalo]

Per ogni *cr*: Crociera, *it*: Itinerario tali che *(cr, it): cr_it,*

- sia S insieme di tutti *sc*: Scalo tali che
  - *(it, sc): it_sc*
  - *(cr, sc): cr_sc*
- sia sc_primo tale che sc_primo in S AND sc_primo.relativo_arrivo non è valorizzato
- sia sc_ultimo tale che sc_ultimo in S AND sc_ultimo.relativa_partenza non è valorizzato
- devono essere veri seguenti condizioni:
  - *sc_primo.relativa_partenza.giorno* = 0
  - per ogni *sc* in S tale che *sc ≠ sc_primo* e *sc ≠ sc_ultimo*
    - sc.*relativo_arrivo* deve essere valorizzato
    - *sc.relativo_partenza* deve essere valorizzato

[V.Crociera.NoOverbooking]

Per ogni c: Crociera
c.num_posti_liberi() >= 0

Operazioni di classe Cliente

eta(): Intero > 0

precondizioni:

- *this.data_nascita* < adesso

postcondizioni:

- operazione non fa side-effect
- viene restituito il valore "*result*" definito come segue:
  - num_anni = *adesso.anno* - *this.data_nascita.anno*

  result = num_anni

Operazioni di classe Crociera

data_fine_programmata(): Data

precondizioni: nessuna
postcondizioni:

- operazione non fa side-effect
- viene restituito il valore "*result*" definito come segue:
    - sia *it*: Itinerario tale che *(this, it): cr_it*
    - sia data_fine = *this.data_inizio_programmata.data* + **(***it.durata_programmata_giorni()* ***in giorni)***

  result = data_fine

data_fine_effettiva(): DataOra

precondizioni: nessuna
postcondizioni:

- operazione non fa side-effect
- viene restituito il valore "*result*" definito come segue:
    - sia S insieme di tutti *sc*: Scalo tali che
        - *(sc, it): it_sc*, *it*: Itinerario
        - *(this, it): cr_it*
    - sia max il valore massimo tra tutti *sc.numero* in S
    - sia *sc_fin*: Scalo in S tale che *sc.numero* = max

  result = *(this, sc_fin).effettivo_arrivo()*

num_posti_liberi(): Intero >= 0

precondizioni: nessuna
postcondizioni:

- operazione non fa side-effect
- viene restituito il valore "*result*" definito come segue:
    - sia tot = *nav.num_posti* dove *nav*: Nave tale che *(nav, this): nav_cr*
    - sia P insieme di tutti *pren*: Prenotazione tali che *(pren, this): pren_cr*
    - sia sum_pren è la somma di tutti *pren.num_posti* **per ogni** p in P
        - $\sum_{p \in P} p.num_posti$

  result = tot - sum_pren

destinazioni(): DestinazioneScalo [0..*]

precondizioni: nessuna
postcondizioni:

- operazione non fa side-effect
- viene restituito il valore "*result*" definito come segue:
    - sia S insieme dei *sc*: Scalo tali che
        - *(sc, it): it_sc*, *it*: Itinerario
        - *(this, it): cr_it*
    - sia D insieme di *dest*: DestinazioneScalo tali che
        - *(dest, sc): sc_dect* dove *sc* in S

  result = D

Operazioni di classe CrocieraLunaDiMiele

tradizionale(): Booleano

precondizione: nessuna
postcondizioni:

- operazione non fa side-effect
- viene restituito il valore "*result*" definito come segue:
    - sia S insieme di tutti *sc*: Scalo tali che
        - *(this, it): cr_it*, *it*: Itinerario
        - *(it, sc): it_sc*
    - sia DR sottoinsieme di S con *sc* tali che
        - *(sc, dest): sc_dest*, *dest*: DestinazioneScalo
        - *(dest, tip): dest_tip*, *tip*: TipoDestinazione
        - *tip.nome* = "Romantica"
    - sia DD sottoinsieme di S con *sc* tali che
        - *(sc, dest): sc_dest*, *dest*: DestinazioneScalo
        - *(dest, tip): dest_tip*, *tip*: TipoDestinazione
        - *tip.nome* = "Divertente"

  result = |DR| >= |DD|

Operazioni di classe Itinerario

durata_programmata_giorni(): Intero > 0

precondizione:

- esiste almeno un *sc*: Scalo tale che *(this, sc): it_sc*

postcondizione:

- operazione non fa side-effect
- viene restituito il valore "*result*" definito come segue:
    - sia S l'insieme di tutti *sc*: Scalo tali che *(this, sc): it_sc*
    - sia min il valore minimo di sc.numero di tutti *sc* in S
    - sia *sc_iniz*: Scalo tale che *sc_iniz.numero* = min
    - sia max il valore massimo di *sc.numero* di tutti sc in S
    - sia *sc_fin*: Scalo tale che *sc_fin.numero* = max
    - sia durata_giorni = *sc_fin.relativo_arrivo.giorno* - *sc_iniz.relativa_partenza.giorno*

  result = durata_giorni

Operazoni di classe cr_sc

Tipo GiornoOra: Tipo record con campi
giorno: Intero >= 0
ora: Ora

effettivo_arrivo(): DataOra

precondizione: nessuna
postcondizione:

- operazione non fa side-effect
- il risultato "*result*" è definito cosi:
    - sia cr: Crociera e *sc*: Scalo tali che *(cr, sc): this*

  versione 1
    - sia data_arrivo = *cr.data_inizio_programmata*
      aumentata di *sc.relattivo_arrivo.giorno* giorni

aumentata di *sc.relativo_arrivo.ore* ore
aumentata di *this.ritardo_arrivo_minuti* minuti

versione 2

- sia data_eff = *cr.data_inizio_programmata.data* + *sc.relativo_arrivo.giorno*
- sia ora_eff = *cr.data_inizio_programmata.ora* + *sc.relativo_arrivo.ora* + *(this.ritardo_arrivo_minuti / 60 in ore)*
  - se ora_eff >= 24
    - data_eff viene aumentata 1 giorno
    - ora_eff viene diminuita a 24 ore
- sia *data_arrivo*: DataOra tale che
  - *data_arrivo.data* = data_eff
  - *data_arrivo.ora* = ora_eff

result = data_arrivo

effettiva_partenza(): DataOra

precondizione: nessuna
postcondizione:

- operazione non fa side-effect
- il risultato "*result*" è definito cosi:
  - sia *cr*: Crociera e *sc*: Scalo tali che *(cr, sc): this*

versione 1

- sia data_partenza = *cr.data_inizio_programmata*
  aumentata di *sc.relattiva_partenza.giorno* giorni
  aumentata di *sc.relativa_partenza.ore* ore
  aumentata di *this.ritardo_partenza_minuti* minuti

versione 2

- sia data_eff = *cr.data_inizio_programmata.data* + *sc.relattiva_partenza.giorno*
- sia ora_eff = *cr.data_inizio_programmata.ora* + *sc.relattiva_partenza.ora* + *(this.ritardo_pertenza_minuti / 60 in ore)*
  - se ora_eff >= 24
    - data_eff viene aumentata 1 giorno
    - ora_eff viene diminuita a 24 ore
- sia data_partenza: DataOra tale che
  - *data_partenza.data* = data_eff
  - *data_partenza.ora* = ora_eff

result = data_partenza

Operazioni di classe DestinazioneScalo

Operazioni ausiliarie

destinazione_esotica(): Boolean

precondizione: nessuna
postcondizione:

- operazione non fa side-effect
- *result* = True se e solo se
  - esiste *naz*: Nazione tale che
    - *naz.continente* =/= "Europa"
    - *(naz, ref): naz_reg*, *reg*: Regione
    - *(reg, cit): reg_cit*, *cit*: Citta

        - *(cit, this): cit_dest*

crociere_in_periodo(d_in: DataOra, d_fin: DataOra): Crociera[0..*]

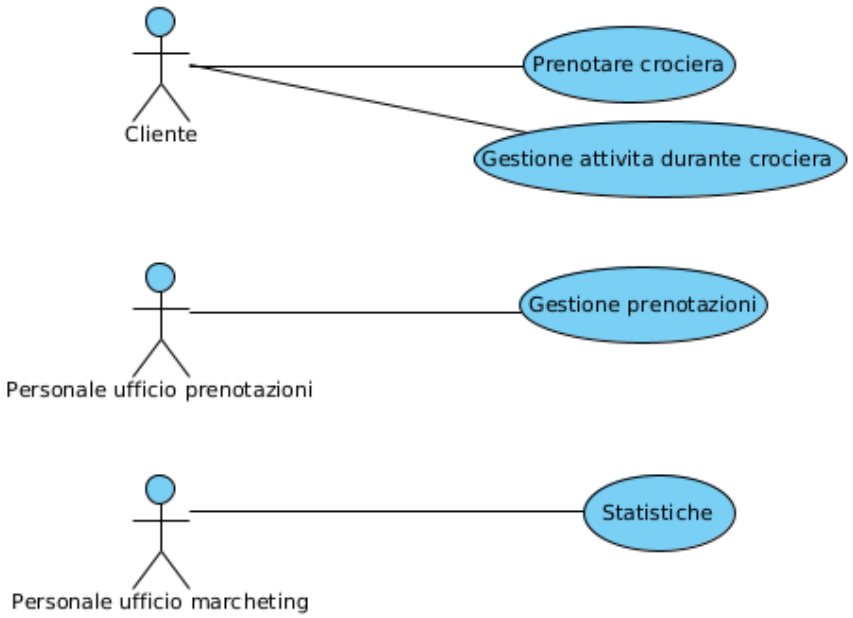
precondizione: nessuna
postcondizione:

- operazione non fa side-effect
- il risultato "*result*" è definito come segue
    - sia S insieme di tutti *sc*: Scalo tali che
        - *(sc, this): sc_dest*
    - sia C insieme di cr: Crociera tali che
        - *(cr, sc): cr_sc,* dove *sc* in S
    - sia CS insieme di link *(cr, st): cr_sc* dove *cr* in C e *sc* in S
    - sia R è sottoinsieme di C con *cr*: Crociera tali che
        - d_in <= *(cr, sc).effettivo_arrivo()* <= d_fin

        oppure
        - d_in <= *(cr, sc).effettiva_pertenza()* <= d_fin

result = R

1.2. Diagramma UML degli Use-Case



1.2.1. Operazioni dello use-case Gestione prenotazioni

1. Dato un cliente che desidera prenotare un certo numero di posti per una crociera c, il personale dell'Ufficio Prenotazioni deve poter effettuare la relativa prenotazione. La richiesta di prenotazione deve essere rifiutata nel caso il numero di posti disponibili, all'istante corrente, per la crociera c non sia sufficiente.

prenota_crociera(cl: Cliente, cr: Crociera, n: Intero > 0): Prenotazione

precondizioni:

- n <= *cr.numero_posti_liberi()*

postcondizioni:

- viene creato e restituito oggetto *pren*: Prenotazione con valori
    - *pren.istante* = adesso
    - *pren.num_posti* = n
- viene creato il link *(pren, cl): cl_pren*
- viene creato il link *(pren, cr): pren_cr*

1.2.2. Operazioni dello use-case Statistiche

2. L'Ufficio Marketing deve poter calcolare l'età media dei clienti che hanno prenotato, in un dato periodo, almeno una crociera che prevede una destinazione esotica (ovvero che si trova in un continente diverso dall'Europa)

eta_media_clienti_in_destinazione_esotica(d_in: DataOra, d_fin: DataOra): Intero > 0

precondizioni:

- esiste almeno un oggetto pren: Prenotazione tale che
    - d_in <= *pren.istante* <= d_fin
    - (*pren*, *cr*): *pren_cr*, *cr*: Crociera tale che
        - in *cr.destinazioni()* esiste *dest*: Destinazione tale che *dest.esotica* = True

postcondizioni

- operazione non fa side-effect
- viene restituito il valore "*result*" definito come segue:
    - sia C insieme di *cr*: Crociere tali che
        - in *cr.destinazioni()* esiste *dest*: Destinazione tale che *dest.esotica* = True
    - sia E insieme di tutti cl: Clienti tali che
        - *(cl, pren): cl_pren*, *pren*: Prenotazione tale che
            - d_in <= *pren.istante* <= d_fin
        - *(pren, cr): pren_cr* dove *cr* in C
    - sia sum è la somma di tutti i valori *cl.eta()* per cl in E

  result = sum / |E|

3. L'Ufficio Marketing deve poter calcolare la **percentuale** delle destinazioni da considerarsi gettonate in un periodo dato. Una destinazione va considerata gettonata in un certo periodo se è stata raggiunta, in quel periodo, da almeno dieci crociere di luna di miele, oppure da almeno quindici crociere per famiglie

percentuale_destinazioni_gettonate(d_in: DataOra, d_fin: DataOra): Reale in 0..1

precondizione:

- d_in < d_fin

postcondizione:

- viene restituito il valore "*result*" definito come segue:
    - Sia D insieme di dest: DestinazioneScalo tali che
        - |*dest.crociere_in_periodo(d_in, d_fin)*| > 0
    - se |D| > 0:
        - Sia DG insieme delle coppie *(dest, g)* dove
            - *dest*: DestinazioneScalo in D
            - *g* è il valore che viene definito in questo modo:
                - sia C insieme di *cr*: Crociera tali che C = *dest.crociere_in_periodo(d_in, d_fin)*
                - se |C| = 0 # non serve, in D ci sono solo quelli dest per quali che |C| > 0
                - *g* = False
                - oppure
                - sia CLM insieme di tutti *cr*: Crociera in C che sono anche *cr*: CrocieraLunaDiMiele
                - sia CF insieme di tutti *cr*: Crociera in C che sono anche *cr*: CrocieraPerFamiglia
                - se e solo se |CLM| >= 10 oppure |CF| >= 15
                - *g* = True
                - altrimenti
                - *g* = False
        - Sia DGT sottoinsieme delle coppie *(dest, g)* di DG con g = True

        - Sia n = |DGT| / |DG|
    - se |D| = 0
        - n = 0

result = n

1.2.3. Operazioni dello use-case Gestione attività durante crociera

posti_da_vedere(dest: DestinazioneScalo): PostoEscursione[0..*]

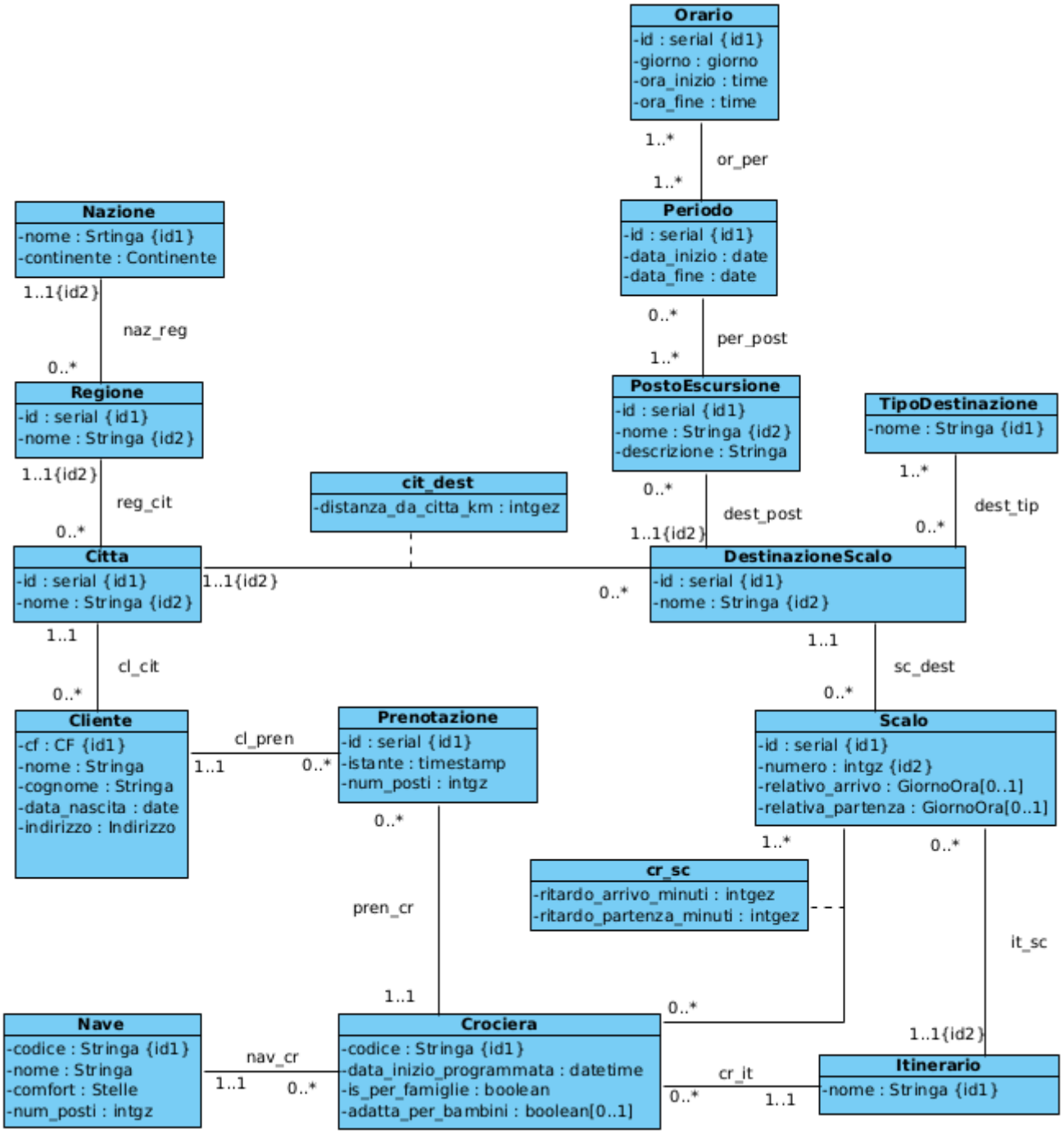
precondizioni: nessuna
postcondizioni:

- operazione non fa side-effect
- viene restituito il valore "*result*" definito come segue:
    - Sia P insieme di tutti *post*: PostoEscursione tali che *(post, dest): dest_post*

result = P

2. Ristrutturazione per Basi Dati

2.1. Diagramma UML ristrutturato per bd



2.2. Specifica dei vincoli esterni

[V.Crociera.Disjoint]
Per ogni cr: Crociera

- p.is_per_famiglie = True se e solo se p.adatta_per_bambini è valorizzato

2.3. Traduzione diretta

2.3.1. Tipi di dato

```
create domain stringa as varchar;
```

```sql
create domain intgz as integer check (value > 0);

create domain intgez as integer check (value >= 0);

create domain cf as
   char(16)
   check (value ~ '^[A-Z]{6}[0-9]{2}[A-Z][0-9]{2}[A-Z][0-9]{3}[A-Z]$');

create domain stelle as integer
   check (value BETWEEN 3 and 5);

create domain stringa_not_null as
   varchar(100) check(value is not null);

create domain cap_not_null as
   char(5) check (value is not null and value ~ '^[0-9]{5}$');

create type indirizzo as (
   via     stringa_not_null,
   civico  stringa_not_null,
   cap      cap_not_null
);

create domain intgez_not_null as
   integer check (value is not null and value >= 0);

create domain time_not_null as
   time check (value is not null);

create type giornoOra as (
   giorno  intgez_not_null,
   ora     time_not_null
);

create type giorno as
   enum('Lunedi', 'Martedi', 'Mercoledi', 'Giovedi', 'Venerdi', 'Sabato', 'Domenica');

create type continente as
   enum('Africa', 'America', 'Asia', 'Europa', 'Oceania', 'Antartide');
```

2.3.2. Schema relazionale

```sql
create table nazione (
   nome stringa primary key
);

create table regione (
   id serial primary key,
```

```sql
   nome stringa not null,
   nazione stringa not null,
   unique (nome, nazione),
   foreign key (nazione)
       references nazione(nome)   -- accorpa naz_reg
);

create table citta (
   id serial primary key,
   nome stringa not null,
   regione integer not null,
   unique (nome, regione),
   foreign key (regione)
       references regione(id)    -- accorpa reg_cit
);

create table cliente (
   cf cf primary key,
   nome stringa not null,
   cognome stringa not null,
   data_nascita date not null,
   indirizzo indirizzo not null,
   citta integer not null,
   foreign key (citta)
       references citta(id)      -- accorpa cl_cit
);

create table nave (
   codice stringa primary key,
   nome stringa not null,
   comfort stelle not null,
   num_posti intgz not null
);

create table itinerario (
   nome stringa primary key
);

create table tipoDestinazione (
   nome stringa primary key
);

create table orario (
   id serial primary key,
   giorno giorno not null,
   ora_inizio time not null,
   ora_fine time not null
   -- v.inclusione orario(id) occorre in or_per(orario)
);
```

```sql
create table periodo (
   id serial primary key,
   data_inizio date not null,
   data_fine date not null
   -- v.inclusione periodo(id) occorre in or_per(orario)
   -- v.inclusione peiodo(id) occorre in per_post(periodo)
);

create table or_per (
   orario integer not null,
   periodo integer not null,
   primary key (orario, periodo),
   foreign key (orario)
       references orario(id),
   foreign key (periodo)
       references periodo(id)
);

create table destinazioneScalo (
   id serial primary key,
   nome stringa not null,
   citta integer not null,
   distanza_da_citta_km intgez not null default 0,
   unique (nome, citta),
   foreign key (citta)
       references citta(id)      -- accorpa cir_dest
   -- v.inclusione destinazionescalo(id) occorrei in dest_tip(destinazioneScalo)
);

create table dest_tip (
   tipoDestinazione stringa not null,
   destinazioneScalo integer not null,
   primary key (tipoDestinazione, destinazioneScalo),
   foreign key (tipoDestinazione)
       references tipoDestinazione(nome),
   foreign key (destinazioneScalo)
       references destinazioneScalo(id)
);

create table postoEscursione (
   id serial primary key,
   nome stringa not null,
   destinazioneScalo integer not null,
   unique (nome, destinazioneScalo),
   descrizione stringa not null,
   foreign key (destinazioneScalo)
       references destinazioneScalo(id)    -- accora dest_post
);

create table per_post (
```

```
   periodo integer not null,
   postoEscursione integer not null,
   primary key (periodo, postoEscursione),
   foreign key (periodo)
       references periodo(id),
   foreign key (postoEscursione)
       references postoEscursione(id)
);

create table scalo (
   id serial primary key,
   numero intgez not null,
   itinerario stringa not null,
   unique (numero, itinerario),
   -- relativo_arrivo giornoOra,
   -- relativa_partenza giornoOra,
   -- spezzo giornoOra in 2 valori separati per controllare i tipi uguali
   rel_arr_giorno intgez,  -- giornoOra.giorno
   rel_arr_ora time,       -- giornoOra.ora
   rel_part_giorno intgez, -- giornoOra.giorno
   rel_part_ora time,      -- giornoOra.giorno
   destinazioneScalo integer not null,
   foreign key (itinerario)
       references itinerario(nome),    -- accorpa it_sc
   foreign key (destinazioneScalo)
       references destinazioneScalo(id),

   check (
       (rel_arr_giorno is null and rel_arr_ora is null)
       or
       (rel_arr_giorno is not null and rel_arr_ora is not null)
   ),
   check (
       (rel_part_giorno is null and rel_part_ora is null)
       or
       (rel_part_giorno is not null and rel_part_ora is not null)
   ),
   check (
       not (
           rel_arr_giorno is null
           and rel_arr_ora is null
           and rel_part_giorno is null
           and rel_part_ora is null
           )
   ),
   check (
       rel_arr_giorno is null
       or rel_arr_ora is null
       or rel_part_giorno is null
       or rel_part_ora is null
```

```
        -- se tutte 4 non sono null allora deve essere vero che
        or (
            rel_arr_giorno < rel_part_giorno
            or (
                rel_arr_giorno = rel_part_giorno
                and rel_arr_ora < rel_part_ora
            )
        )
    )
);

create table crociera (
    codice stringa primary key,
    data_inizio_programmata timestamp not null,
    is_per_famiglie boolean not null,
    adatta_per_bambini boolean,
    nave stringa not null,
    itinerario stringa not null,
    foreign key (nave)
        references nave(codice),       -- accorpa nav_cr
    foreign key (itinerario)
        references itinerario(nome),    -- accorpa cr_it
    -- v.inclusione crociera(codice) occorre in cr_sc(crociera)
    check ((is_per_famiglie = TRUE and adatta_per_bambini is not null)
        or
        (is_per_famiglie = FALSE and adatta_per_bambini is null)
        )
);

create table cr_sc (
    ritardo_arrivo_minuti intgez not null,
    ritardo_partenza_minuti intgez not null,
    scalo integer not null,
    crociera stringa not null,
    primary key (scalo, crociera),
    foreign key (scalo)
        references scalo(id),
    foreign key (crociera)
        references crociera(codice)
);

create table prenotazione (
    id serial primary key,
    istante timestamp not null,
    num_posti intgz not null,
    cliente cf not null,
    crociera stringa not null,
    foreign key (cliente)
        references cliente(cf),        -- accorpa cl_pren
    foreign key (crociera)
```

```
        references crociera(codice)    -- accorpa pren_cr
);
```